

ملخص مشروع: Smart Research Assistant

1. الفكرة والهدف (Project Overview)

نظام مساعد بحث ذكي (RAG System) متكامل. يقوم النظام بقراءة ملفات PDF، فهمها، والإجابة على الأسئلة منها حصراً مع ذكر رقم الصفحة والمصدر، مع رفض الإجابة إذا كانت المعلومة غير موجودة لمنع هلوسة الذكاء الاصطناعي وتأليف الإجابات.

2. التقنيات المستخدمة (Tech Stack)

- **FastAPI & Uvicorn:** بناء الخادم الخلفي (Backend) ومسارات الـ API (upload/ و query/).
- **Streamlit:** واجهة المستخدم التفاعلية للدردشة ورفع الملفات (Frontend).
- **ChromaDB & HuggingFace:** قاعدة بيانات المتجهات لتخزين النصوص المقطعة والبحث السريع.
- **Google Gemini (gemini-3.6-flash):** نموذج الذكاء الاصطناعي لصياغة الردود.
- **LangChain & pdfplumber:** استخراج النصوص بدقة مع الاحتفاظ بأرقام الصفحات.

3. هيكلية الملفات (Directory Structure)

```
smart-research-assistant/  
├── api/  
│   └── routes.py  
├── data/  
│   ├── chroma_db/  
│   └── raw_pdfs/  
├── src/  
│   ├── config.py  
│   ├── ingestion.py  
│   ├── vectorstore.py  
│   └── rag_engine.py  
├── ui/  
│   └── app.py  
├── main.py  
└── requirements.txt
```

4. الأكواد المصدرية (Source Code)

1. الإعدادات العامة (src/config.py)

```
from pathlib import Path  
  
BASE_DIR = Path(__file__).resolve().parent.parent
```

```
DATA_DIR = BASE_DIR / "data"
RAW_PDFS_DIR = DATA_DIR / "raw_pdfs"
CHROMA_DIR = DATA_DIR / "chroma_db"

EMBEDDING_MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
CHUNK_SIZE = 800
CHUNK_OVERLAP = 150
```

2. `src/ingestion.py` (PDF قراءة وتقطيع ال)

```
import pdfplumber
from typing import List
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter
from src.config import CHUNK_SIZE, CHUNK_OVERLAP

def extract_text_with_pages(pdf_path: str) -> List[Document]:
    documents = []
    with pdfplumber.open(pdf_path) as pdf:
        for page_idx, page in enumerate(pdf.pages):
            text = page.extract_text()
            if text:
                documents.append(
                    Document(
                        page_content=text,
                        metadata={"source": pdf_path.split("/")[-1].split("\")[0],
"page": page_idx + 1}
                    )
                )
    return documents

def chunk_documents(docs: List[Document]) -> List[Document]:
    splitter = RecursiveCharacterTextSplitter(chunk_size=CHUNK_SIZE,
chunk_overlap=CHUNK_OVERLAP)
    return splitter.split_documents(docs)
```

3. `src/vectorstore.py` (قاعدة المتجهات)

```
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma
from src.config import CHROMA_DIR, EMBEDDING_MODEL_NAME

def get_embedding_function():
    return HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL_NAME)

def get_vectorstore():
    return Chroma(
```

```

        persist_directory=str(CHROMA_DIR),
        embedding_function=get_embedding_function()
    )

def store_documents_in_chroma(chunks):
    vectorstore = get_vectorstore()
    vectorstore.add_documents(chunks)
    vectorstore.persist()
    return vectorstore

```

4. `src/rag\_engine.py` (محرك الإجابة)

```

import os
from typing import Dict, Any, List
from dotenv import load_dotenv
import google.generativeai as genai
from langchain_core.documents import Document
from src.vectorstore import get_vectorstore

load_dotenv()
genai.configure(api_key=os.getenv("GOOGLE_API_KEY"))

def format_docs(docs: List[Document]) -> str:
    formatted = []
    for doc in docs:
        source = doc.metadata.get("source", "Unknown")
        page = doc.metadata.get("page", "?")
        formatted.append(f"[Source: {source} | Page: {page}]\n{doc.page_content}")
    return "\n\n".join(formatted)

def ask_research_assistant(question: str, top_k: int = 3) -> Dict[str, Any]:
    vectorstore = get_vectorstore()
    retriever = vectorstore.as_retriever(search_kwargs={"k": top_k})
    matched_docs = retriever.invoke(question)

    context = format_docs(matched_docs)
    sources = [{"source": d.metadata.get("source"), "page": d.metadata.get("page")} for d
in matched_docs]

    prompt = f"""You are an expert research assistant. Answer the question accurately
using ONLY the provided context.
If the answer cannot be found in the context, say clearly: "I cannot find this information
in the provided document."

Context:
{context}

Question: {question}

```

Helpful Answer:""

```
model = genai.GenerativeModel("gemini-3.6-flash")
response = model.generate_content(prompt)
return {"answer": response.text, "sources": sources}
```

5. `api/routes.py` (مسارات السيرفر)

```
import os, shutil
from fastapi import APIRouter, UploadFile, File, HTTPException
from pydantic import BaseModel
from typing import List, Dict, Any
from src.ingestion import extract_text_with_pages, chunk_documents
from src.vectorstore import store_documents_in_chroma
from src.rag_engine import ask_research_assistant

router = APIRouter()
UPLOAD_DIR = "data/raw_pdfs"
os.makedirs(UPLOAD_DIR, exist_ok=True)

class QueryRequest(BaseModel):
    question: str
    top_k: int = 3

class QueryResponse(BaseModel):
    answer: str
    sources: List[Dict[str, Any]]

@router.post("/upload")
async def upload_pdf(file: UploadFile = File(...)):
    if not file.filename.endswith(".pdf"):
        raise HTTPException(status_code=400, detail="Only PDF files are supported.")
    file_path = os.path.join(UPLOAD_DIR, file.filename)
    with open(file_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)
    try:
        docs = extract_text_with_pages(file_path)
        chunks = chunk_documents(docs)
        store_documents_in_chroma(chunks)
        return {"filename": file.filename, "status": "Indexed successfully",
"chunks_count": len(chunks)}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/query", response_model=QueryResponse)
async def query_rag(request: QueryRequest):
    try:
        return ask_research_assistant(request.question, top_k=request.top_k)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

6. `main.py` (نقطة التشغيل)

```
from fastapi import FastAPI
from api.routes import router

app = FastAPI(title="Smart Research Assistant API", version="1.0.0")
app.include_router(router, prefix="/api")

@app.get("/")
def read_root():
    return {"message": "Smart Research Assistant API is running!"}
```

7. `ui/app.py` (واجهة المستخدم - Streamlit)

```
import streamlit as st
import requests

API_BASE_URL = "http://127.0.0.1:8000/api"

st.set_page_config(page_title="Smart Research Assistant", page_icon="📖", layout="wide")
st.title("📖 Smart Research Assistant")

with st.sidebar:
    st.header("📄 Upload Document")
    uploaded_file = st.file_uploader("Select a PDF file", type=["pdf"])
    if uploaded_file and st.button("Index Document", use_container_width=True):
        files = {"file": (uploaded_file.name, uploaded_file.getvalue(), "application/pdf")}
        res = requests.post(f"{API_BASE_URL}/upload", files=files)
        if res.status_code == 200:
            st.success(f"Indexed! Chunks: {res.json().get('chunks_count')}")
        else:
            st.error(res.text)

if "messages" not in st.session_state:
    st.session_state.messages = []

for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])
        if msg.get("sources"):
            with st.expander("📌 Citations & Sources"):
                for s in msg["sources"]:
                    st.markdown(f"- **Source:** `{s.get('source')}` | **Page:** `{s.get('page')}`")

if prompt := st.chat_input("Ask a question about your documents..."):
    st.session_state.messages.append({"role": "user", "content": prompt})
```

```

with st.chat_message("user"):
    st.markdown(prompt)

with st.chat_message("assistant"):
    res = requests.post(f"{API_BASE_URL}/query", json={"question": prompt, "top_k":
3})

    if res.status_code == 200:
        data = res.json()
        st.markdown(data.get("answer"))
        if data.get("sources"):
            with st.expander("📌 Citations & Sources"):
                for s in data["sources"]:
                    st.markdown(f"- **Source:** `{s.get('source')}` | **Page:**
`s.get('page')`")
                st.session_state.messages.append({"role": "assistant", "content":
data.get("answer"), "sources": data.get("sources")})

```